CMPE 48A Term Project – Phase 1 Report

Cloud-Native Deployment of *Local History Story Map* on Google Cloud Platform

November 2026

1. Team

Role	Name
Developer	Aysu Keskin
Developer	Musa Kaan Güney

2. Application Code

Repository: https://github.com/AysuKeskin/storymap-cloud-native

The application is *Local History Story Map*, a platform where users discover and publish location-based community stories through a map, a feed, a timeline and full-text search. We reuse the application code that was developed earlier in the CMPE 354 software-engineering course (original upstream: https://github.com/bounswe/bounswe2026group4). The cloud-native design, the Kubernetes manifests, Terraform code, Cloud Functions and Locust tests are new work for this course.

Layer	Technology
Backend API	Django 4.2 + Django REST Framework, Gunicorn, JWT auth
Web frontend	React + Vite, served by nginx
Database	MySQL 8.0
Already containerised	Dockerfiles for backend and nginx/frontend, Docker Compose

Main API domains: users and authentication, stories (map / feed / timeline / search), media uploads, comments and likes, tags, notifications, gamification (badges) and a geocoding proxy to OpenStreetMap Nominatim.

Changes planned to make the application cloud-native

1. Move media storage from the local disk to **Cloud Storage** with `django-storages`, so that API pods are stateless and can be scaled freely.
2. Replace the per-process in-memory cache with a shared **Redis** cache. This also gives a distributed rate-limit lock for the Nominatim proxy, which today works only within one process.
3. Move e-mail sending out of the request path. The API publishes an event to **Pub/Sub** and a **Cloud Function** sends the e-mail.
4. Add **image thumbnail generation** as a Cloud Function triggered by uploads to the bucket.
5. Read all configuration from Kubernetes Secrets and ConfigMaps, and run database migrations as a Kubernetes `Job`.

3. Tentative Cloud Architecture

3.1 Overview

The system runs in one GCP project, and Terraform creates all of its resources. It has four layers:

- **Entry:** a load balancer is the only public endpoint.
- **Compute (Kubernetes):** the frontend and the Django API run on GKE. The API scales automatically with HPA and shares a Redis cache.
- **Data (VM + storage):** MySQL runs on a Compute Engine VM, and media files are stored in Cloud Storage. Pods keep no state, so they can be added or removed at any time.
- **Background (serverless):** Cloud Functions create thumbnails and send e-mails (via Pub/Sub), so this slow work does not delay API responses.

This covers the three required building blocks: Kubernetes, virtual machines and Cloud Functions. Locust sends load from a separate project, through the same public endpoint as real users.

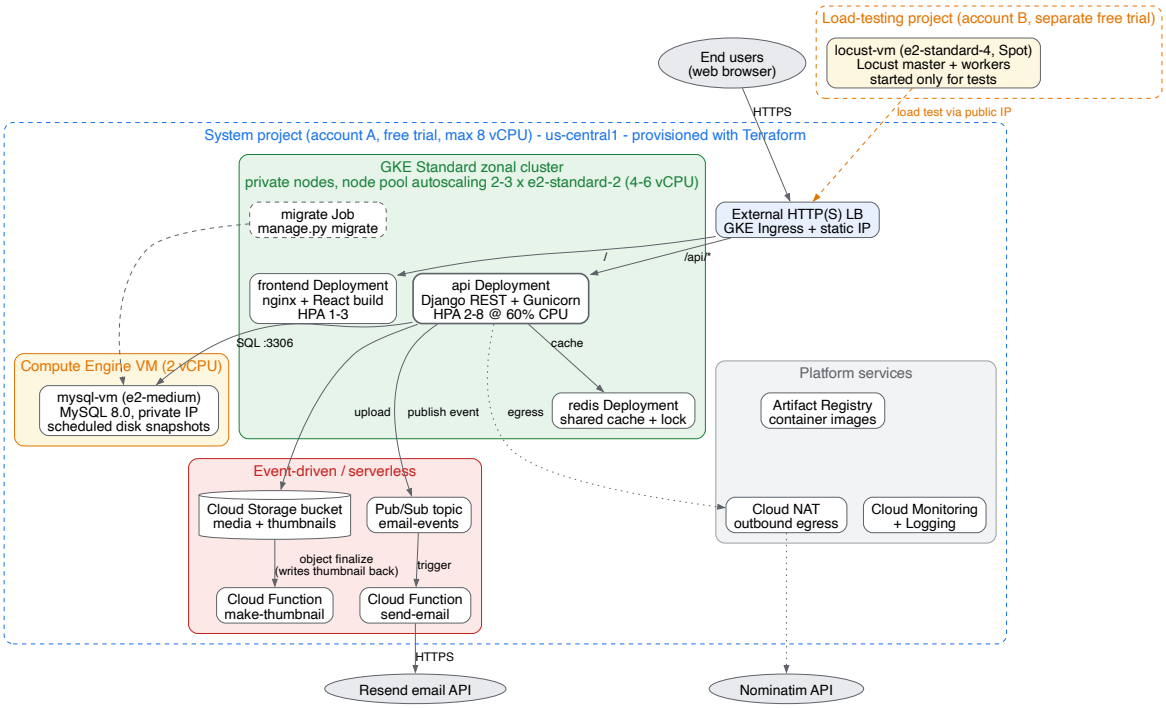


Figure 1: Tentative GCP architecture. The system runs in one project that is provisioned with Terraform. The load generator runs in a separate project.

3.2 Platform constraints and how the design meets them

- **8 vCPU limit.** A GCP free-trial account can run at most **8 Compute Engine vCPUs at the same time**. GKE nodes count towards this limit because they are Compute Engine VMs. The whole system is therefore sized to stay at or below 8 vCPUs, even at peak (Table 1).
- **Time window.** The trial credit of the system account expires on **23 December 2026**. All development, experiments and the demo are planned to finish before that date. Because the infrastructure is written in Terraform, it can be recreated in another account if the schedule slips.

- **Separate load-testing project.** The Locust load generator is not part of the system under test. It runs in a **separate project under the partner's free-trial account** and reaches the system only through the public load-balancer IP, the same way real users do. This keeps the 8 vCPUs of the system account for the application. It also keeps the load generator from competing for CPU with the components it measures.
- **Node type.** With at most three nodes allowed, we use **e2-standard-2** nodes (2 dedicated vCPUs, 8 GB) instead of shared-core e2-medium. They give more allocatable CPU per node and stable CPU measurements, without shared-core burst effects, for the HPA experiments.

Project	**Compute Engine resources**	**vCPUs (limit 8)**
System (account A)	GKE node pool 2–3 × e2-standard-2	4–6
	mysql-vm, 1 × e2-medium	2
	Peak total	**8**
Load testing (account B)	locust-vm, 1 × e2-standard-4 (only during tests)	4

Table 1: vCPU budget per free-trial account. Cloud Functions, Cloud Storage, Pub/Sub and the load balancer do not count towards the Compute Engine limit.

3.3 Components

Component	**GCP service**	**Role in the system**
Load balancer	External HTTP(S) LB (created by GKE Ingress), static IP	Single public entry point. Routes `/api/*` to the API service and `/` to the frontend.
frontend	GKE Deployment (+ HPA 1–3)	nginx serving the compiled React app. Lightweight and stateless.
api	GKE Deployment (+ HPA 2–8 at 60 % CPU)	Django REST API on Gunicorn. Main scaling target and the focus of the performance tests.
redis	GKE Deployment + ClusterIP Service	Shared cache for hot read endpoints (`/stories/map`, `/stories/feed`, `/stories/timeline`), geocoding results and a distributed lock. Runs in-cluster instead of Memorystore to save budget.
migrate	Kubernetes Job	Runs `manage.py migrate` before each rollout.
mysql-vm	Compute Engine VM (e2-medium)	MySQL 8.0 on a private IP, reachable only from inside the VPC. Persistent disk with a scheduled snapshot policy. Chosen over Cloud SQL for cost and to satisfy the VM requirement with a stateful, functional role.
locust-vm	Compute Engine VM (e2-standard-4, Spot) in the separate load-testing project	Runs Locust master and workers. Started only during experiments. Generates load from outside the system project, through the public endpoint.
Media bucket	Cloud Storage	Stores user-uploaded images, audio and video, plus generated thumbnails.
make-thumbnail	Cloud Function, GCS *object finalize* trigger	Creates resized thumbnails for every uploaded image and writes them back to the bucket.

Component	GCP service	Role in the system
email-events	Pub/Sub topic	Asynchronous queue that decouples the API from e-mail delivery (registration verification, password reset, notification e-mails).
send-email	Cloud Function, Pub/Sub trigger	Consumes `email-events` and sends the message through the Resend e-mail API. Retries on failure.
Artifact Registry	Docker repository	Stores the `api` and `frontend` images built by GitHub Actions.
Cloud NAT	Cloud Router + NAT	Outbound internet access for private GKE nodes and the MySQL VM (Nominatim, package installs). Nothing in the VPC has a public IP.
Monitoring / Logging	Cloud Monitoring, Cloud Logging	CPU, memory, pod count, LB latency and error metrics used in the performance evaluation.

3.4 Interactions

1. **Read path.** A request goes Client → LB → `api` pod. The pod checks Redis first. On a cache miss it queries MySQL on the VM and stores the result in Redis with a short TTL.
2. **Write path (story with media).** The `api` pod writes the story row to MySQL and streams the file to Cloud Storage. The bucket's *finalize* event triggers `make-thumbnail`, which writes the thumbnail next to the original.
3. **Asynchronous e-mail.** Registration and notification flows publish a small JSON message to `email-events` and return immediately. `send-email` then delivers the message. A slow e-mail provider no longer slows down API requests.
4. **Scaling.** The HPA adds `api` pods (2 → 8) when average CPU exceeds 60 %. The GKE cluster autoscaler adds a third node when pods cannot be scheduled. Three nodes are the maximum allowed by the 8 vCPU limit.
5. **Deployment.** Terraform creates the VPC, NAT, GKE cluster, MySQL VM, bucket, Pub/Sub topic, Cloud Functions, Artifact Registry and a billing budget in the system project. A separate small Terraform configuration creates `locust-vm` in the load-testing project. Kubernetes manifests (Deployments, Services, HPA, Ingress, Job) are applied with `kubectl`.

4. Cost Estimation

Assumptions. Prices are list prices for `us-central1` taken from the GCP pricing pages, and will be re-checked in the GCP Pricing Calculator before deployment. The project runs from **13 November to 23 December 2026** (40 days, 960 hours). The billing currency of the account is TRY. The trial credit of TRY 14,440.57 corresponds to $300, so we use 1 USD ≈ 48.1 TRY. On 13 November, **TRY 13,796 (≈ $287)** of credit remains. About $13 was used before this project started.

4.1 System project (account A): baseline for 40 days, always on

Resource	Configuration	Unit price	Cost (960 h)
GKE management fee	1 zonal Standard cluster	$0.10/h ($96)	**$0.00** (covered by the $74.40/month GKE free-tier credit)

Resource	Configuration	Unit price	Cost (960 h)
GKE nodes	2 × e2-standard-2	\$0.067/h each	\$128.64
Node boot disks	2 × 30 GB pd-standard	\$0.04/GB-month	\$3.20
mysql-vm	1 × e2-medium	\$0.0335/h	\$32.16
mysql-vm disk + snapshots	20 GB pd-balanced + ∼5 GB snapshots	\$0.10 and \$0.05/GB-month	\$3.00
HTTP(S) Load Balancer	1 forwarding rule	\$0.025/h	\$24.00
Cloud NAT	3 VM endpoints + low traffic	\$0.0014/h per VM + \$0.045/GB	\$4.53
Cloud Storage, Artifact Registry	∼2 GB each	\$0.020 and \$0.10/GB-month	\$0.30
Cloud Functions	< 100k invocations	free tier: 2M req., 180k vCPU-s	\$0.00
Pub/Sub	< 1 GB	first 10 GiB/month free	\$0.00
Monitoring and Logging	within free allotment (50 GiB logs)	–	\$0.00
Internet egress (normal use)	∼10 GB	∼\$0.12/GB	\$1.50
Total (40 days)			**≈ \$197**

4.2 Load-testing campaign (about 30 test hours)

Billed to	Resource	Usage	Cost
System (A)	Third GKE node from autoscaling	30 h × \$0.067/h	\$2.05
System (A)	LB data processing	∼50 GB × \$0.008/GB	\$0.40
System (A)	Response egress to the load generator	∼50 GB × up to \$0.12/GB	\$6.00
Load testing (B)	locust-vm e2-standard-4	30 h × \$0.134/h (on-demand; Spot is cheaper)	\$4.02
Load testing (B)	locust-vm disk + egress	20 GB pd-standard, ∼5 GB requests	\$1.70
Total			**≈ \$14**

4.3 Project total vs. the available credit

Scenario	Calculation	Total	Share of credit
System (A), worst case: always on	\$197 + \$8.45 (load tests)	**≈ \$206**	72 % of \$287
System (A), planned: torn down when idle	∼50 % of baseline hours + load tests	**≈ \$107**	37 % of \$287
Load testing (B)	locust-vm only	**≈ \$6**	2 % of \$300
Both accounts combined, worst case	\$206 + \$6	**≈ \$212**	below one \$300 credit

Even if both accounts are counted against a single \$300 budget, the worst case stays below the limit.

Cost-control measures

- `terraform destroy` when the system is not needed (nights, weekends). Everything can be recreated from code in about 15 minutes.
- `locust-vm` exists only during experiments and uses **Spot** pricing (∼60–90 % cheaper than on-demand).
- The node pool is capped at 3 nodes, which also enforces the 8 vCPU limit. Boot disks are 30 GB pd-standard instead of the 100 GB default. GKE Managed Prometheus is disabled.
- Redis and MySQL are self-hosted, which avoids Memorystore (∼$35+/month) and Cloud SQL (∼$25–50/month).
- Terraform creates a billing budget (`google_billing_budget`) with **alerts at 50 %, 75 % and 90 %**. After each teardown we check the billing report for leftover resources such as load balancers, disks, static IPs and snapshots.